

CodSoft AI Internship - Task 5 Report

Task Name: Face Detection and Recognition (MTCNN) | **Domain:** Artificial Intelligence

Submitted by: Prince Maurya

Institution: IILM University

Roll Number: 25SCS1003001538 | **Section:** 2CSE7

Date: September 20, 2026

1. Objective & Problem Overview

Develop a robust, deep learning-based computer vision application to detect human faces and facial landmarks in uploaded images. The system must preprocess raw image data, execute high-accuracy detections, and extract cropped regions of interest for individual faces[cite: 2].

2. System Architecture & Methodology

The application implements the Multi-task Cascaded Convolutional Network (MTCNN) framework[cite: 2]. The preprocessing pipeline automatically resizes large images (scaling down widths exceeding 1600px) and applies mild sharpening using Gaussian blur and weighted image addition[cite: 2]. The detector converts the enhanced BGR image to RGB format before extracting bounding boxes and facial landmarks[cite: 2]. Detections are strictly filtered to ensure high accuracy, requiring a minimum 90% confidence threshold and a minimum face size of 30x30 pixels[cite: 2]. Validated faces are highlighted with bounding boxes and saved as individual cropped JPEG files into a dedicated output directory[cite: 2].

3. Core Python Implementation

```
import cv2
import os
from mtcnn import MTCNN

# 1. Preprocessing (Resize and Sharpen)
def preprocess_image(image, max_width=1600):
    original_height, original_width = image.shape[:2]
    if original_width > max_width:
        scale = max_width / original_width
        image = cv2.resize(image, (int(original_width * scale), int(original_height * scale)))

    blurred = cv2.GaussianBlur(image, (0, 0), 1.0)
    return cv2.addWeighted(image, 1.3, blurred, -0.3, 0)

# 2. Initialization and Format Conversion
detector = MTCNN()
image = cv2.imread(image_path)
processed_image = preprocess_image(image)
rgb_image = cv2.cvtColor(processed_image, cv2.COLOR_BGR2RGB)

# 3. Deep Learning Detection
detections = detector.detect_faces(rgb_image)
valid_faces = []

# 4. Filtering (90% Confidence, 30px Min Size)
for face in detections:
    confidence = float(face["confidence"])
    x, y, w, h = face["box"]
    if confidence >= 0.90 and w >= 30 and h >= 30:
        valid_faces.append(face)

# 5. Cropping and Landmarking
os.makedirs("detected_faces", exist_ok=True)
for i, face in enumerate(valid_faces):
    x, y, w, h = face["box"]

    # Save cropped face
    face_crop = processed_image[max(0, y):y+h, max(0, x):x+w]
    cv2.imwrite(f"detected_faces/face_{i+1}.jpg", face_crop)

# Draw Bounding Box & Landmarks
```

```
cv2.rectangle(processed_image, (x, y), (x+w, y+h), (0, 255, 0), 3)
for name, point in face["landmarks"].items():
    cv2.circle(processed_image, point, 5, (0, 0, 255), -1)

cv2.imwrite("output_faces_mtcnn.jpg", processed_image)
```

4. System Evaluation & Execution Output

```
=====
                        FACE DETECTION REPORT
=====
Input image           : uploaded_sample.jpg
Image width           : 1920px
Image height          : 1080px
Raw detections        : 5
Valid faces           : 3
Confidence threshold  : 90%
Minimum face size     : 30px
Detection time        : 1.204 seconds
Average confidence    : 98.45%
Output file           : output_faces_mtcnn.jpg
=====
```